

```
clc, clear variables

%% Bodeplot Settings
% Preferences
set(cstprefs.tbxprefs, 'MagnitudeUnits', 'dB');
set(cstprefs.tbxprefs, 'FrequencyUnits', 'Hz');
set(cstprefs.tbxprefs, 'UnwrapPhase', 'Off');
set(cstprefs.tbxprefs, 'Grid', 'On');
set(groot, 'defaultLineLineWidth', 1.2); % global for all new lines

% Bodeoptions
opt = bodeoptions('cstprefs');
opt.Xlim = { [0.1 450] }; % x-axis: 0.1 Hz to 1000 Hz

%% Sampling Time
Ts = 125 * 1.0e-6; % Gyro loop
Ts_cntr = 2 * Ts; % Control loop
Ts_log = 2 * Ts_cntr; % Logging loop (not used here, just info)
fs = 1/Ts;

t = (0:Ts:10).'; % Time vector from 0s to 10s (column vector)
N = length(t); % number of samples

% frequency vector in Hz
f = (0:N-1)*(fs/N);

s = tf('s');
z = tf('z');

%% Chirp Signal (logarithmic)
f0 = 0.1; f1 = 500; % Hz
u = chirp(t, f0, 10, f1, 'logarithmic', 0); % numerical vector
U_In = timeseries(u, t); % optional: for Simulink/Scopes

%% Interference Signals

% Input interference (white noise, non-periodic)
A_l = 0.4;
omega_l = 2*pi*10; % kept for compatibility, not used
l = A_l * randn(size(t)); % non-periodic input noise
Int_In = timeseries(l, t); % optional: for Simulink/Scopes

% Output interference (band-limited noise, non-periodic)
A_n = 0.4;
omega_n = 2*pi*20; % kept for compatibility, not used
raw_noise = randn(size(t));
[b_n, a_n] = butter(4, [5 200]/(fs/2), 'bandpass'); % limit to 5-200 Hz band
n = A_n * filtfilt(b_n, a_n, raw_noise);
Int_Out = timeseries(n, t); % optional: for Simulink/Scopes

%% Combined Input Signals
% (For MATLAB calculations use numeric vectors; timeseries above only for Simulink)
u_tilde = u + l;
```

```
%% Force everything into column form
[t, u, l, n, u_tilde] = deal(t(:), u(:), l(:), n(:), u_tilde(:));

%% Transfer function Plant
P = (s+10) / ((s + 1)*(s+2));

%% Output Signal
y_tilde = lsim(P, u_tilde, t);    % system response
y        = y_tilde + n;           % add output interference

%% Plot of Input
figure(1)
subplot(311)
plot(t,u);grid on;
title('Input u(t)');
subplot(312)
plot(t,l);grid on;
title('Input Interference Signal l(t)');
subplot(313)
plot(t,u_tilde);grid on;
title('Input u-tilde(t)');
sgtitle('Combination Input Signals')

%% Plot of Output
figure(2)
subplot(311)
plot(t,y_tilde);grid on;
title('Output y-tilde(t)');
subplot(312)
plot(t,n);grid on;
title('Output Interference Signal n(t)');
subplot(313)
plot(t,y);grid on;
title('Output y(t)');
sgtitle('Combination Output Signals')

%% Plot of Signals
figure(3)
subplot(411)
plot(t,u);grid on;
title('Input u(t)');
subplot(412)
plot(t,u_tilde);grid on;
title('Input u-tilde(t)');
subplot(413)
plot(t,y_tilde);grid on;
title('Output y-tilde(t)');
subplot(414)
plot(t,y);grid on;
title('Output y(t)');
sgtitle('Input and Output Signals')

%% Transferfunction without adjustments (plain FFT)
U = fft(u);
```

```

Y      = fft(y);
H1     = Y ./ U;

% use only positive frequencies up to Nyquist
H1 = H1(1:floor(N/2));
f   = f(1:floor(N/2));

% create FRD object for bode plot
Hfrd = frd(H1, 2*pi*f); % bode expects rad/s
figure(4)
h = bodeplot(P, Hfrd, opt);
grid on
legend('G(s) (true)', 'Estimated (FFT)')
op = getoptions(h);
op.Title.String = '';
setoptions(h, op);
sgtitle('Transfer Function Y(s)/U(s)');

%% Transferfunctions with Signalenergy (single-shot on full record)
Syy = Y .* conj(Y);
Suu = U .* conj(U);
Syu = Y .* conj(U);

th   = 1e-15 * max(Suu); % get rid of extremely small numbers for numerical ✓
errors
H2   = Syu ./ Suu;
H2(Suu < th) = NaN; % guard
H2 = H2(1:floor(N/2));

% create FRD object for bode plot
figure(5)
h5 = bodeplot(P, Hfrd, opt);
grid on
legend('P(s) (true)', 'Estimated (FFT)')
op5 = getoptions(h5);
op5.Title.String = '';
setoptions(h5, op5);
sgtitle('Transfer Function Syu/Suu');

%% APPLY_ROTFILTFILT (inline, from existing data)

% Basisband low-pass (zero-phase)
%fc = 5; % adjust if sweep speed changes
%[b_lp, a_lp] = butter(4, fc/(fs/2));

% Phase (sinarg) of the logarithmic chirp from f0,f1 and total duration
Ttot = t(end) - t(1);
r     = f1 / f0;
phi   = 2*pi*f0 * ( Ttot/log(r) ) * ( r.^((t - t(1))/Ttot) - 1 ); % Also know as ✓
sinarg

% Phasors
p     = exp(1i*phi);
pc    = conj(p);

```

```

% Input/Output signals (use actual plant input & measured output)
X_in = u - mean(u);
X_out = y - mean(y);

% Rotate forward
in_R = X_in .* p;   in_Q = X_in .* pc;
out_R = X_out .* p;   out_Q = X_out .* pc;

% Zero-phase low-pass in baseband
% in_R = filtfilt(b_lp, a_lp, in_R);   in_Q = filtfilt(b_lp, a_lp, in_Q);
% out_R = filtfilt(b_lp, a_lp, out_R);   out_Q = filtfilt(b_lp, a_lp, out_Q);

% Back-rotate & recombine (real)
inp = real(0.5*(in_R .* pc + in_Q .* p));   % demodulated, filtered input
out = real(0.5*(out_R .* pc + out_Q .* p));   % demodulated, filtered output

%% Simple FRF from rotated signals via FFT (optional quick look)
U_rotFFT = fft(inp);
Y_rotFFT = fft(out);
H_rotFFT = Y_rotFFT ./ U_rotFFT;
H_rotFFT = H_rotFFT(1:floor(N/2));
f_axis = (0:floor(N/2)-1)*(fs/N);
Hfrd_rotFFT = frd(H_rotFFT, 2*pi*f_axis);

figure(9)
p9 = bodeplot(P, Hfrd_rotFFT, opt);
grid on
legend('P(s) (true)', 'Estimated (FFT, rotated)')
op9 = getoptions(p9); op9.Title.String = ''; setoptions(p9, op9);
sgtitle('Transfer Function (FFT on rotated signals)');

% Transfer function according to Welch (original & rotated via function)

% Use same parameters you already used above
Nest = round(2 / Ts);   % number of samples per segment
overlap = 0.9;   % 50% overlap

% Welch FRF for original signals u -> y
[Hfrd_welch, ~] = welch_frf(u, y, Nest, overlap, fs, true);

% Welch FRF for rotated (Lock-in) signals inp -> out
[Hfrd_welch_rot, ~] = welch_frf(inp, out, Nest, overlap, fs, true);

% Plots (kept consistent with your style)
figure(6); clf
p6 = bodeplot(P, Hfrd_welch, Hfrd, opt);
grid on
legend('P(s) (true)', 'Welch Signal Energie', 'Signal Energie', ...
      'Location','SouthWest');
title('Transfer Function (Original, Welch)');

figure(7)

```

```

p7 = bodeplot(P, Hfrd_welch_rot, Hfrd_rotFFT, opt);
grid on
legend('P(s) (true)', 'Welch Signal (rotated)', 'Signal rotated', ...
      'Location','SouthWest');
op7 = getoptions(p7); op7.Title.String = ''; setoptions(p7, op7);
sgtitle('Transfer Function (Rotated/Lock-in, Welch)');

figure(10)
p10 = bodeplot(P, Hfrd_welch, Hfrd_welch_rot, opt);
grid on
legend('P(s) (true)', 'Welch H_2 (orig)', 'Welch H_2 (rotated)', ...
      'Location','SouthWest');
op10 = getoptions(p10); op10.Title.String = ''; setoptions(p10, op10);
sgtitle('Welch: Original vs Rotated');

%% Welch fuction
function [Hfrd_welch, freq] = welch_frf(u_sig, y_sig, Nest, overlap, fs, use_hann)
% WELCH_FRF Estimate FRF via Welch method (H2 = S_yu / S_uu) and coherent mean ✓
(Y/U).
% Inputs:
%   u_sig   : input signal (column vector)
%   y_sig   : output signal (column vector)
%   Nest    : samples per segment
%   overlap : fraction [0..1] of overlap (e.g., 0.5)
%   fs      : sampling frequency [Hz]
%   use_hann: if true, use Hann window; else rectangular
%
% Outputs:
%   Hfrd_welch : FRD object of H2 estimator
%   Hfrd_coh   : FRD object of coherent estimator mean(Y/U)
%   freq       : frequency vector [Hz] (0..Nyquist)

    delta = 0 * var(u_sig);      % small regularization

    u_sig = u_sig(:); y_sig = y_sig(:);      %Conversion from row vector to column ✓
vector

    if use_hann                    % Decide witch window will be used
        w = hann(Nest, 'periodic');
    else
        w = ones(Nest,1);
    end

    Ndata = size(u_sig, 1);
    Noverlap = floor(overlap * Nest);

    % When you apply a window (like a Hann window), the signal energy is reduced
    % This the factor to compensate that
    denom = sum(w) / Nest / 2;

    % frequency axis (Hz) and single-sided length
    freq = (0:Nest/2).' * (fs / Nest);
    Nfreq = length(freq);

```

```

% global mean removal
u_sig = u_sig - mean(u_sig);
y_sig = y_sig - mean(y_sig);

% Welch average of single-sided spectra [Suu, Syu, Syy]
Pavg = zeros(Nfreq, 3);
Navg = 0;

ind_start = 1;
ind_end   = Nest;
Ndelta    = Nest - Noverlap;

while ind_end <= Ndata
    ind = ind_start:ind_end;

    u_inp = u_sig(ind);
    y_out = y_sig(ind);

    % per-segment mean removal
    u_seg = u_inp - mean(u_inp);
    y_seg = y_out - mean(y_out);

    % window
    u_seg = u_seg .* w;
    y_seg = y_seg .* w;

    % FFT with single-sided normalization
    U = fft(u_seg) / (Nest * denom);
    Y = fft(y_seg) / (denom * Nest);

    % two-sided -> single-sided (0..Nyquist), then fix DC/Nyquist
    Pact = [U.*conj(U), Y.*conj(U), Y.*conj(Y)];
    Pls   = Pact(1:Nfreq, :);
    Pls(1, :) = Pls(1, :) / 4;
    Pls(end, :) = Pls(end, :) / 4;

    Pavg = Pavg + Pls;
    Navg = Navg + 1;

    % Next segment
    ind_start = ind_start + Ndelta;
    ind_end   = ind_end + Ndelta;
end

if Navg > 0
    Pavg = Pavg / Navg;
end

Suu = Pavg(:,1);
Syu = Pavg(:,2);

H_welch = Syu ./ (Suu + delta); % H2 estimator with regularization

```

```

    Hfrd_welch = frd(H_welch, freq, 1/fs, 'Units','Hz');
end

%% Add function form Michi

addpath ../lib/

% Linear filter for zero phase excitation filter (apply_rotfiltfilt)
Dlp = sqrt(3) / 2;
wlp = 2 * pi * 10;
Glp = c2d(tf(wlp^2, [1 2*Dlp*wlp wlp^2]), Ts_log, 'tustin');

% Lock-in clean-up (rotate, zero-phase filter in baseband, back-rotate)
u_rf = apply_rotfiltfilt(Glp, phi, u); % filtered input
y_rf = apply_rotfiltfilt(Glp, phi, y); % filtered output

% Welch parameters (use same scale as your script; ensure even Nest)
Nest = round(4 / Ts); % number of samples per segment
NoverlapS = round(0.9 * Nest); % 90% overlap
win = hann(Nest, 'periodic');
delta = 1e-12*var(u); % tiny regularization for Suu

% FRF of Variant C (function-based)
[G_C, C_C, freq_Hz, ~] = estimate_frequency_response(u_rf, y_rf, win, NoverlapS, ✓
Nest, Ts, delta);

% --- Bode overlay: true P(s) vs. Original (Welch) vs. Variant C ---
figure(11)
pC = bodeplot(P, Hfrd_welch_rot, G_C, opt); grid on
legend('P(s) (true)', 'Janick', 'Michi', ...
'Location', 'SouthWest');
opC = getoptions(pC); opC.Title.String = ''; setoptions(pC, opC);
sgtitle('Janick vs Michi');

```